A reinforcement learning extension to the Almgren-Chriss framework for optimal trade execution

Dieter Hendricks
School of Computational and
Applied Mathematics
University of the Witwatersrand
Johannesburg, South Africa
Email: dieter.hendricks@students.wits.ac.za

Diane Wilcox
School of Computational and
Applied Mathematics
University of the Witwatersrand
Johannesburg, South Africa
Email: diane.wilcox@wits.ac.za

***Abstract*—Reinforcement learning is explored as a candidate machine learning technique to enhance existing analytical solutions for optimal trade execution with elements from the market microstructure. Given a volume-to-trade, fixed time horizon and discrete trading periods, the aim is to adapt a given volume trajectory such that it is dynamic with respect to favourable/unfavourable conditions during realtime execution, thereby improving overall cost of trading. We consider the standard Almgren-Chriss model with linear price impact as a candidate base model. This model is popular amongst sell-side institutions as a basis for arrival price benchmark execution algorithms. By training a learning agent to modify a volume trajectory based on the market's prevailing spread and volume dynamics, we are able to improve post-trade implementation shortfall by up to 10.3% on average compared to the base model, based on a sample of stocks and trade sizes in the South African equity market.**

I. Introduction

A critical problem faced by participants in investment markets is the so-called optimal liquidation problem, viz. how best to trade a given block of shares at minimal cost. Here, cost can be interpreted as in Perold's implementation shortfall ([21]), i.e. adverse deviations of actual transaction prices from an arrival price baseline when the investment decision is made. Alternatively, cost can be measured as a deviation from the market volume-weighted trading price (VWAP) over the trading period, effectively comparing the specific trader's performance to that of the average market trader. In each case, the primary problem faced by the trader/execution algorithm is the compromise between price impact and opportunity cost when executing an order.

Price impact here refers to adverse price moves due to a large trade size absorbing liquidity supply at available levels in the order book (temporary price impact). As market participants begin to detect the total volume being traded, they may also adjust their bids/offers downward/upward to anticipate order matching (permanent price impact) [16]. To avoid price impact, traders may split a large order into smaller child orders over a longer period. However, there may be exogenous market forces which result in execution at adverse prices (opportunity cost). This behaviour of institutional investors was empirically demonstrated in [9], where they observed that typical trades of large investment management firms are almost always broken up into smaller trades and executed over the course of a day or several days.

Several authors have studied the problem of optimal liquidation, with a strong bias towards stochastic dynamic programming solutions. See [7], [17], [26], [1] as examples. In this paper, we consider the application of a machine learning technique to the problem of optimal liquidation. Specifically we consider a case where the popular Almgren-Chriss closed-form solution for a trading trajectory (see [1]) can be enhanced by exploiting microstructure attributes over the trading horizon using a reinforcement learning technique.

Reinforcement learning in this context is essentially a calibrated policy mapping states to optimal actions. Each state is a vector of observable attributes which describe the current configuration of the system. It proposes a simple, model-free mechanism for agents to learn how to act optimally in a controlled Markovian domain, where the quality of action chosen is successively improved for a given state [10]. For the optimal liquidation problem, the algorithm examines the salient features of the current order book and current state of execution in order to decide which action (e.g. child order price or volume) to select to service the ultimate goal of minimising cost.

The first documented large-scale empirical application of reinforcement learning algorithms to the problem of optimised trade execution in modern financial markets was conducted by [20]. They set up their problem as a minimisation of implementation shortfall for a buying/selling program over a fixed time horizon with discrete time periods. For actions, the agent could choose a price to repost a limit order for the remaining shares in each discrete period. State attributes included elapsed time, remaining inventory, current spread, immediate cost and signed volume. In their results, they found that their reinforcement learning algorithm improved the execution efficiency by 50% or more over traditional submit-

Authorized licensed use limited to: Columbia University Libraries. Downloaded on October 23,2026 at 22:57:41 UTC from IEEE Xplore. Restrictions apply.

and-leave or market order policies.

Instead of a pure reinforcement learning solution to the problem, as in [20], we propose a hybrid approach which *enhances* a given analytical solution with attributes from the market microstructure. Using the Almgren-Chriss (AC) model as a base, for a finite liquidation horizon with discrete trading periods, the algorithm determines the proportion of the AC-suggested trajectory to trade based on prevailing volume/spread attributes. One would expect, for example, that allowing the trajectory to be more aggressive when volumes are relatively high and spreads are tight may reduce the ultimate cost of the trade. In our implementation, a static volume trajectory is preserved for the duration of the trade, however the proportion traded is dynamic with respect to market dynamics. As in [20], a market order is executed at the end of the trade horizon for the remaining volume, to ensure complete liquidation. An important consideration in our analysis is the specification of the problem as a finite-horizon Markov Decision Process (MDP) and the consequences for optimal policy convergence of the reinforcement learning algorithm. In [20], they use an approximation in their framework to address this issue by incorporating elapsed time as a state attribute, however they do not explicitly discuss convergence. We will use the findings of [14] in our model specification and demonstrate near-optimal policy convergence of the finite-horizon MDP problem.

The model described above is compared with the base Almgren-Chriss model to determine whether it increases/decreases the cost of execution for different types of trades consistently and significantly. This study will help determine whether reinforcement learning is a viable technique which can be used to extend existing closed-form solutions to exploit the nuances of the microstructure where the algorithms are applied.

This paper proceeds as follows: Section 2 introduces the standard Almgren-Chriss model. Section 3 describes the specific hybrid reinforcement learning technique proposed, along with a discussion regarding convergence to optimum action values. Section 4 discusses the data used and results, comparing the 2 models for multiple trade types. Section 5 concludes and proposes some extensions for further research.

II. The Almgren-Chriss model

Bertsimas and Lo are pioneers in the area of optimal liquidation, treating the problem as a stochastic dynamic programming problem [7]. They employed a dynamic optimisation procedure which finds an explicit closed-form best execution strategy, minimising trading costs over a fixed period of time for large transactions. Almgren and Chriss extended the work of [7] to allow for risk aversion in their framework [1]. They argue that incorporating the uncertainty of execution of an optimal solution is consistent with a trader's utility function. In particular, they employ a price process which permits linear permanent and temporary price impact functions to construct an efficient frontier of optimal execution. They define a trading strategy as being *efficient* if there is no strategy which has lower execution cost variance for the same or lower level of expected execution cost.

The exposition of their solution is as follows: They assume that the security price evolves according to a discrete arithmetic random walk:

$$S_k = S_{k-1} + \sigma\tau^{1/2}\xi_k - \tau g(\frac{n_k}{\tau}),$$

where:

$S_k =$ price at time k,
$\sigma =$ volatility of the security,
$\tau =$ length of discete time interval,
$\xi_k =$ draws from independent random variables,
$n_k =$ volume traded at time k and
$g(.) =$ permanent price impact.

Here, permanent price impact refers to changes in the equilibrium price as a direct function of our trading, which persists for at least the remainder of the liquidation horizon. Temporary price impact refers to adverse deviations as a result of absorbing available liquidity supply, but where the impact dissipates by the next trading period due to the resilience of the order book. Almgren and Chriss introduce a temporary price impact function $h(v)$ to their model, where $h(v)$ causes a temporary adverse move in the share price as a function of our trading rate v [1]. Given this addition, the actual security transaction price at time k is given by:

$$\tilde{S_k} = S_{k-1} - h(\frac{n_k}{\tau}).$$

Assuming a *sell* program, we can then define the total trading revenue as:

$$\sum_{k=1}^{N} n_k\tilde{S_k} = XS_0 + \sum_{k=1}^{N}(\sigma\tau^{1/2}\xi_k - \tau g(\frac{n_k}{\tau}))x_k - \sum_{k=1}^{N} n_k h(\frac{n_k}{\tau}),$$

where $x_k = X - \sum_{j=1}^{k} n_j = \sum_{j=k+1}^{N} n_j$ for $k = 0, 1, ..., N$.

The total cost of trading is thus given by $x = XS_0 - \sum n_k\tilde{S_k}$, i.e. the difference between the target revenue value and the total actual revenue from the execution. This definition refers to Perold's implementation shortfall measure (see [21]), and serves as the primary transaction cost metric which is minimised in order to maximise trading revenue. Since implementation shortfall is a random variable, Almgren and Chriss compute the following:

$$E(x) := \sum_{k=1}^{N} \tau x_k g(\frac{n_k}{\tau}) + \sum_{k=1}^{N} n_k h(\frac{n_k}{\tau})$$

and

$$Var(x) := \sigma^2 \sum_{k=1}^{N} \tau {x_k}^2.$$

The distribution of implementation shortfall is Gaussian if the ξ_k are Gaussian.

Authorized licensed use limited to: Columbia University Libraries. Downloaded on October 23,2026 at 22:57:41 UTC from IEEE Xplore. Restrictions apply.

Given the overall goal of minimising execution costs and the variance of execution costs, they specify their objective function as:

$$\min_x \{E(x) + \lambda Var(x)\},$$

where:

$$x = \text{ implementation shortfall,}$$
$$\lambda = \text{ level of risk aversion.}$$

The intuition of this objective function can be thought of as follows: Consider a stock which exhibits high price volatility and thus a high risk of price movement away from the reference price. A risk averse trader would prefer to trade a large portion of the volume immediately, causing a (known) price impact, rather than risk trading in small increments at successively adverse prices. Alternatively, if the price is expected to be stable over the liquidation horizon, the trader would rather split the trade into smaller sizes to avoid price impact. This trade-off between speed of execution and risk of price movement is what governs the shape of the resulting trade trajectory in the AC framework.

A detailed derivation of the general solution can be found in [1]. Here, we state the general solution:

$$x_j = \frac{\sinh(\kappa(T - t_j))}{\sinh(\kappa T)} X \text{ for } j = 0, ..., N.$$

The associated trade list is:

$$n_j = \frac{2\sinh(\frac{1}{2}\kappa\tau)}{\sinh(\kappa T)} \cosh(\kappa(T - t_{j-\frac{1}{2}}))X \text{ for } j = 0, ..., N,$$

where:

$$\kappa = \frac{1}{\tau}\cosh^{-1}\left(\frac{\tau^2}{2}\tilde{\kappa}^2 + 1\right),$$
$$\tilde{\kappa}^2 = \frac{\lambda\sigma^2}{\eta(1 - \frac{\rho\tau}{2\eta})},$$
$$\eta = \text{temporary price impact parameter,}$$
$$\rho = \text{permanent price impact parameter,}$$
$$\tau = \text{length of discrete time period.}$$

This implies that for a program of selling an initially long position, the solution decreases monotonically from its initial value to zero at a rate determined by the parameter κ. If trading intervals are short, κ^2 is essentially the ratio of the product of volatility and risk-intolerance to the temporary transaction cost parameter. We note here that a larger value of κ implies a *more rapid* trading program, again conceptually confirming the propositions of [17] that an intolerance for execution risk leads to a larger concentration of quantity traded early in the trading program. Another consequence of this analysis is that different sized baskets of the same securities will be liquidated in the same manner, barring scale differences and provided the risk aversion parameter λ is held constant. This may be counter-intuitive, since one would expect larger baskets to be effectively less liquid, and thus follow a *less rapid* trading program to minimise price impact costs.

It should be noted that the AC solution yields a suggested volume trajectory over the liquidation horizon, however there is no discussion in [1] as to the prescribed *order type* to execute the trade list. We have assumed that the trade list can be executed as a series of *market orders*. Given that this implies we are always crossing the spread, one needs to consider that traversing an order book with thin volumes and widely-spaced prices could have a significant transaction cost impact. We thus consider a reinforcement learning technique which learns *when* and *how much* to cross the spread, based on the current order book dynamics.

The general solution outlined above assumes linear price impact functions, however the model was extended by Almgren in [2] to account for non-linear price impact. This extended model can be considered as an alternative base model in future research.

III. A REINFORCEMENT LEARNING APPROACH

The majority of reinforcement learning research is based on a formalism of Markov Decision Processes (MDPs) [4]. In this context, reinforcement learning is a technique used to numerically solve for a calibrated policy mapping states to optimal or near-optimal actions. It is a framework within which a learning agent repeatedly observes the state of its environment, and then performs a chosen action to service some ultimate goal. Performance of the action has an immediate numeric reward or penalty and changes the state of the environment [10]. The problem of solving for an optimal policy mapping states to actions is well-known in stochastic control theory, with a significant contribution by Bellman [5]. Bellman showed that the computational burden of an MDP can be significantly reduced using what is now known as dynamic programming. It was however recognised that two significant drawbacks exist for classical dynamic programming: Firstly, it assumes that a complete, known model of the environment exists, which is often not realistically obtainable. Secondly, problems rapidly become computationally intractable as the number of state variables increases, and hence, the size of the state space for which the value function must be computed increases. This problem is referred to as the *curse of dimensionality* [25].

Reinforcement learning offers two advantages over classical dynamic programming: Firstly, agents learn online and continually adapt while performing the given task. Secondly, the methods can employ function approximation algorithms to represent their knowledge. This allows them to generalize across the state space so that the learning time scales much better [12]. Reinforcement learning algorithms do not require knowledge about the exact model governing an MDP and thus can be applied to MDP's where exact methods become infeasible.

Although a number of implementations of reinforcement learning exist, we will focus on *Q-learning*. This is a model-free technique first introduced by [27], which can be used to find the optimal, or near-optimal, action-selection policy for a given MDP.

Authorized licensed use limited to: Columbia University Libraries. Downloaded on October 23,2026 at 22:57:41 UTC from IEEE Xplore. Restrictions apply.

During *Q-learning*, an agent's learning takes place during sequential episodes. Consider a discrete finite world where at each step n, an agent is able to register the current state $x_n \in X$ and can choose from a finite set of actions $a_n \in A$. The agent then receives a probabilistic reward r_n, whose mean value $R_{x_n}(a_n)$ depends only on the current state and action. According to [27], the state of the world changes probabilistically to y_n according to:

$$Prob(y_n = y|x_n, a_n) = P_{x_n y}(a_n).$$

The agent is then tasked to learn the optimal policy mapping states to actions, i.e. one which maximises total discounted expected reward. Under some policy mapping π and discount rate γ ($0 < \gamma < 1$), the value of state x is given by:

$$V^{\pi}(x) = R_x(\pi(x)) + \gamma \sum_y P_{xy}(\pi(x)) V^{\pi}(y).$$

According to [6] and [23], the theory of dynamic programming says there is at least one optimal stationary policy π^* such that

$$V^*(x) = V^{\pi^*}(x) = \max_a \{R_x(a) + \gamma \sum_y P_{xy}(a) V^{\pi^*}(y)\}.$$

We also define $Q^{\pi}(x, a)$ as the expected discounted reward from choosing action a in state x, and then following policy π thereafter, i.e.

$$Q^{\pi}(x, a) = R_x(a) + \gamma \sum_y P_{xy}(\pi(x)) V^{\pi}(y).$$

The task of the *Q-learning* agent is to determine V^*, π^* and Q^{π^*} where $P_{xy}(a)$ is unknown, using a combination of exploration and exploitation techniques over the given domain. It can be shown that $V^*(x) = \max_a Q^*(x, a)$ and that an optimal policy can be formed such that $\pi^*(x) = a^*$. It thus follows that if the agent can find the optimal Q-values, the optimal action can be inferred for a given state x. It is shown in [10] that an agent can learn Q-values via experiential learning, which takes place during sequential episodes. In the nth episode, the agent:

- observes its current state x_n,
- selects and performs an action a_n,
- observes the subsequent state y_n as a result of performing action a_n,
- receives an immediate reward r_n and
- uses a learning factor α_n, which decreases gradually over time.

Q is updated as follows:

$$\begin{aligned} Q(x_n, a_n) \quad = \quad & Q(x_n, a_n) + \\ & \alpha_n(r_n + \gamma \max_b Q(x_{n+1}, b) - Q(x_n, a_n)). \end{aligned}$$

Provided each state-action pair is visited infinitely often, [10] show that Q converges to Q^* for any exploration policy. Singh et al. provide guidance as to specific exploration policies for asymptotic convergence to optimal actions and asymptotic exploitation under the *Q-learning* algorithm, which we incorporate in our analysis [24].

A. Implications of finite-horizon MDP

The above exposition presents an algorithm which guarantees optimal policy convergence of a stationary infinite-horizon MDP. The stationarity assumption, and hence validity of the above result, needs to be questioned when considering a finite-horizon MDP, since states, actions and policies are time-dependent [22]. In particular, we are considering a discrete period, finite trading horizon, which guarantees execution of a given volume of shares. At each decision step in the trading horizon, it is possible to have different state spaces, actions, transition probabilities and reward values. Hence the above model needs revision. Garcia and Ndiaye consider this problem and provide a model specification which suits this purpose [14]. They propose a slight modification to the Bellman optimality equations shown above:

$$V_n^*(x) = \max_{a_n} \{R_x(a_n) + \gamma \sum_y P_{xy}^n(a_{n+1}) V_{n+1}^{\pi^*}(y)\}$$

for all $x \in S_n$, $y \in S_{n+1}$, $a_n \in S_n$, $n \in \{0, 1, ..., N\}$ and $V_{N+1}^*(x) = 0$. This optimality equation has a single solution $V^* = \{V_1^*, V_2^*, ..., V_N^*\}$ that can be obtained using dynamic programming techniques. The equivalent discounted expected reward specification thus becomes:

$$Q_n^{\pi}(x, a_n) = R_x(a_n) + \gamma \sum_y P_{xy}^n(\pi(x)) V_{n+1}^{\pi}(y).$$

They propose a novel transformation of an N-step non-stationary MDP into an infinite-horizon process ([14]). This is achieved by adding an artifical final reward-free absorbing state x_{abs}, such that all actions $a_N \in A_N$ lead to x_{abs} with probability 1. Hence the revised *Q-learning* update equation becomes:

$$Q_{n+1}(x_n, a_n) = Q_n(x_n, a_n) + \alpha_n(x_n, a_n) U_n,$$

where

$$U_n = \begin{cases} r_n + \gamma \max_b Q_n(y_n, b) - Q_n(x_n, a_n) & \text{if } x_n \in S_i, i < N \\ r_n - Q_n(x_n, a_n) & \text{if } x_n \in S_N \\ 0 & \text{otherwise.} \end{cases}$$

If $x_n \notin S_N$, $y_n = x_{n+1}$, otherwise choose randomly in S_1. If $x_{n+1} \in S_j$, select $a_{n+1} \in A_j$. The learning rule for S_N is thus equivalent to setting $V_{N+1}^*(x_{abs}) = Q_{N+1}^*(x_{abs}, a_j) = 0$ $\forall\ a_j \in A_{N+1}$.

Garcia and Ndiaye further show that the above specification (in the case where $\gamma = 1$) will converge to the optimal policy with probability one, provided that each state-action pair is visited infinitely often, $\sum_n \alpha_n(x, a) = \infty$ and $\sum_n \alpha_n^2(x, a) < \infty$ [14].

B. Implementation for optimal liquidation

Given the above description, we are able to discuss our specific choices for state attributes, actions and rewards in the context of the optimal liquidation problem. We need to consider a specification which adequately accounts for our state of execution and the current state of the limit order

Authorized licensed use limited to: Columbia University Libraries. Downloaded on October 23,2026 at 22:57:41 UTC from IEEE Xplore. Restrictions apply.

book, representing the opportunity set for our ultimate goal of executing a volume of shares over a fixed trading horizon.

1) States: We acknowledge that the complexity of the financial system cannot be distilled into a finite set of states and is not likely to evolve according to a Markov process. However, we conjecture that the essential features of the system can be sufficiently captured with some simplifying assumptions such that meaningful insights can still be inferred. For simplicity, we have chosen a look-up table representation of Q. Function approximation variants may be explored in future research for more complex system configurations. As described above, each state $x_n \in X$ represents a vector of observable attributes which describe the configuration of the system at time n. As in [20], we use *Elapsed Time* t and *Remaining Inventory* i as private attributes which capture our state of execution over a finite liquidation horizon T. Since our goal is to modify a given volume trajectory based on favourable market conditions, we include *spread* and *volume* as candidate market attributes. The intuition here is that the agent will learn to increase (decrease) trading activity when *spreads* are narrow (wide) and *volumes* are high (low). This would ensure that a more significant proportion of the total volume-to-trade would be secured at a favourable price and, similarly, less at an unfavourable price, ultimately reducing the post-trade implementation shortfall. Given the look-up table implementation, we have simplified each of the state attributes as follows:

- T = Trading Horizon , V = Total Volume-to-Trade,
- H = Hour of day when trading will begin,
- I = Number of remaining inventory states,
- B = Number of spread states,
- W = Number of volume states,
- sp_n = %ile Spread of the nth tuple,
- vp_n = %ile Bid/Ask Volume of the nth tuple,
- **Elapsed Time**: $t_n = 1, 2, 3, ..., T$,
- **Remaining Inventory**: $i_n = 1, 2, 3, ..., I$,
- **Spread State**: $s_n = \begin{cases} 1, & \text{if } 0 < sp_n \leq \frac{1}{B} \\ 2, & \text{if } \frac{1}{B} < sp_n \leq \frac{2}{B} \\ ... & \\ B, & \text{if } \frac{(B-1)}{B} < sp_n \leq 1, \end{cases}$
- **Volume State**: $v_n = \begin{cases} 1, & \text{if } 0 < vp_n \leq \frac{1}{U} \\ 2, & \text{if } \frac{1}{W} < vp_n \leq \frac{2}{W} \\ ... & \\ W, & \text{if } \frac{(W-1)}{W} < vp_n \leq W. \end{cases}$

Thus, for the nth episode, the state attributes can be summarised as the following tuple:

$$z_n =< t_n, i_n, s_n, v_n > .$$

For sp_n and vp_n, we first construct a historical distribution of spreads and volumes based on the training set. It has been empirically observed that major equity markets exhibit *U*-shaped trading intensity throughout the day, i.e. more activity in mornings and closer to closing auction. A further discussion of these insights can be found in [3] and [8]. In fact, [13] empirically demonstrates that South African stocks exhibit similar characteristics. We thus consider simlulations where training volume/spread tuples are *H*-hour dependent, such that the optimal policy is further refined with respect to trading time (*H*).

2) Actions: Based on the Almgren-Chriss (AC) model specified above, we calculate the AC volume trajectory $(AC_1, AC_2, ..., AC_T)$ for a given volume-to-trade (V), fixed time horizon (T) and discrete trading periods ($t = 1, 2, ..., T$). AC_t represents the proportion of V to trade in period t, such that $\sum_{t=1}^{T} AC_t = V$. For the purposes of this study, we assume that each child order is executed as a *market order* based on the prevailing limit order book structure. We would like our learning agent to modify the AC volume trajectory based on prevailing volume and spread characteristics in the market. As such, the possible actions for our agent include:

- β_j = Proportion of AC_t to trade,
- β_{LB} = Lower bound of volume proportion to trade,
- β_{UB} = Upper bound of volume proportion to trade,
- **Action**: $a_{jt} = \beta_j AC_t$, where $\beta_{LB} \leq \beta_j \leq \beta_{UB}$ and $\beta_j = \beta_{j-1} + \beta_{incr}$.

The aim here is to train the learning agent to trade a higher (lower) proportion of the overall volume when conditions are favourable (unfavourable), whilst still broadly preserving the volume trajectory suggested by the AC model. To ensure that the total volume-to-trade is executed over the given time horizon, we execute any residual volume at the end of the trading period with a *market order*.

3) Rewards: Each of the actions described above results in a volume to execute with a *market order*, based on the prevailing structure of the limit order book. The size of the child order volume will determine how deep we will need to traverse the order book. For example, suppose we have a *BUY* order with a *volume-to-trade* of 20000, split into child orders of 10000 in period t and 10000 in period $t+1$. If the structure of the limit order book at time t is as follows:

- *Level-1 Ask Price* = 100.00; *Level-1 Ask Volume* = 3000
- *Level-2 Ask Price* = 100.50; *Level-2 Ask Volume* = 4000
- *Level-3 Ask Price* = 102.30; *Level-3 Ask Volume* = 5000
- *Level-4 Ask Price* = 103.00; *Level-4 Ask Volume* = 6000
- *Level-5 Ask Price* = 105.50; *Level-5 Ask Volume* = 2000

the volume-weighted execution price will be:

$$\frac{(3000 \times 100) + (4000 \times 100.5) + (3000 \times 102.3)}{10000} = 100.9.$$

Trading more (less) given this limit order book structure will result in a higher (lower) volume-weighted execution price. If the following trading period $t+1$ has the following structure:

- *Level-1 Ask Price* = 99.80; *Level-1 Ask Volume* = 6000
- *Level-2 Ask Price* = 99.90; *Level-2 Ask Volume* = 2000
- *Level-3 Ask Price* = 101.30; *Level-3 Ask Volume* = 7000
- *Level-4 Ask Price* = 107.00; *Level-4 Ask Volume* = 3000
- *Level-5 Ask Price* = 108.50; *Level-5 Ask Volume* = 1000

the volume-weighted execution price for the second child order will be:

$$\frac{(6000 \times 99.8) + (2000 \times 99.9) + (2000 \times 101.3)}{10000} = 100.12.$$

If the reference price of the stock at $t = 0$ is 99.5, then the *implementation shortfall* from this trade is:

$$\frac{((20000 \times 99.5) - (10000 \times 100.9 + 10000 \times 100.12)}{20000 \times 99.5}$$
$$= -0.0101 = -101bps.$$

Authorized licensed use limited to: Columbia University Libraries. Downloaded on October 23,2026 at 22:57:41 UTC from IEEE Xplore. Restrictions apply.

Since the conditions of the limit order book were more favourable for *BUY* orders in period $t+1$, if we had modified the child orders to, say 8000 in period t and 12000 in period $t+1$, the resulting *implementation shortfall* would be:

$$\frac{((20000 \times 99.5) - (8000 \times 100.54 + 12000 \times 100.32)}{20000 \times 99.5}$$
$$= -0.0091 = -91bps.$$

In this example, increasing the child order volume when *Ask Prices* are lower and *Level-1 Volumes* are higher decreases the overal cost of the trade. It is for this reason that *implementation shortfall* is a natural candidate for the rewards matrix in our reinforcement learning system. Each action implies a child order volume, which has an associated volume-weighted execution price. The agent will learn the consequences of each action over the trading horizon, with the ultimate goal of minimising the total trade's *implementation shortfall*.

4) Algorithm and Methodology: Given the above specification, we followed the following steps to generate our results:

- Specify a stock (S), volume-to-trade (V), time horizon (T), and trading datetime (from which the trading hour H is inferred),
- Partition the dataset into independent *training sets* and *testing sets* to generate results (the *training set* always pre-dates the *testing set*),
- Calibrate the parameters for the Almgren-Chriss (AC) volume trajectory (σ, η) using the historical *training set*; set $\rho = 0$, since we assume order book is resilient to trading activity (see below),
- Generate the AC volume trajectory $(AC_1, ..., AC_T)$,
- Train the *Q-matrix* based on the state-action tuples generated by the *training set*,
- Execute the AC volume trajectory at the specified trading datetime (H) on each day in the *testing set*, recording the *implementation shortfall*,
- Use the trained *Q-matrix* to modify the AC trajectory as we execute V at the specified trading datetime, recording the *implementation shortfall* and
- Determine whether the reinforcement learning (RL) model improved/worsened realised *implementation shortfall*.

In order to train the *Q-matrix* to learn the optimal policy mapping, we need to traverse the training data set ($T \times I \times A$) times, where A is the total number of possible actions. The following pseudo-code illustrates the algorithm used to train the *Q-matrix*:

```
Optimal_strategy<V,T,I,A>
   For (Episode 1 to N) {
      Record reference price at t=0
      For t = T to 1 {
         For i = 1 to I
            Calculate episode's STATE attributes <s,v>
            For a = 1 to A {
                Set x = <t,i,s,v>
                Determine the action volume a
                Calculate IS from trade, R(x,a)
                Simulate transition x to y
                Look up max_p Q(y,p)
                Update Q(x,a) = Q(x,a) + alpha*U }}}
Select the lowest-IS action max_p Q(y,p) for optimal policy
```

An important assumption in this model specification is that our trading activity does not affect the market attributes. Although temporary price impact is incorporated into execution prices via depth participation of the *market order* in the prevailing limit order book, we assume the limit order book is resilient with respect to our trading activity. Market resiliency can be thought of as the number of quote updates before the market's spread reverts to its competitive level. Degryse et al. showed that a pure limit order book market (Euronext Paris) is fairly resilient with respect to most order sizes, taking on average 50 quote updates for the spread to normalise following the most aggressive orders [11]. Since we are using 5-minute trading intervals and small trade sizes, we will assume that any permanent price impact effects dissipate by the next trading period. A preliminary analysis of South African stocks revealed that there were on average over 1000 quote updates during the 5-minute trading intervals and the pre-trade order book equilibrium is restored within 2 minutes for large trades. The validity of this assumption however will be tested in future research, as well as other model specifications explored which incorporate permanent effects in the system configuration.

IV. Data and results

A. Data used

For this study, we collected 12 months of market depth tick data (Jan-2012 to Dec-2012) from the Thomson Reuters Tick History (TRTH) database, representing a universe of 166 stocks that make up the South African local benchmark index (ALSI) as at 31-Dec-2012. This includes 5 levels of order book depth (bid/ask prices and volumes) at each tick. The raw data was imported into a MongoDB database and aggregated into 5-minute intervals showing average level prices and volumes, which was used as the basis for the analysis.

B. Stocks, parameters and assumptions

To test the robustness of the proposed model in the South African (SA) equity market we tested a variety of stock types, trade sizes and model parameters. Due to space constraints, we will only show a representative set of results here that illustrate the insights gained from the analysis. The following summarises the stocks, parameters and assumptions used for the results that follow:

- **Stocks**
  - SBK (Large Cap, Financials)
  - AGL (Large Cap, Resources)
  - SAB (Large Cap, Industrials)
- **Model Parameters**
  - β_{LB}: 0, β_{UB}: 2, β_{incr}: 0.25
  - λ: 0.01, τ: 5-min, α_0: 1, γ: 1
  - V: 100 000, 1000 000
  - T: 4 (20-min), 8 (40-min), 12 (60-min)
  - H: 9, 10, 11, 12, 13, 14, 15, 16
  - I, B, W: 5, 10
  - Buy/Sell: BUY
- **Assumptions**
  - Max volume participation rate in order book: 20%
  - Market is resilient to our trading activity

Note, we set $\gamma = 1$ since [14] states that this is a necessary condition to ensure convergence to the optimal policy with

Authorized licensed use limited to: Columbia University Libraries. Downloaded on October 23,2026 at 22:57:41 UTC from IEEE Xplore. Restrictions apply.

probability one for a finite-horizon MDP. We also choose an arbitrary value for λ, although sensitivities to these parameters will be explored in future work. AC parameters are calibrated and *Q-matrix* trained over a 6-month *training set* from 1-Jan-2012 to 30-Jun-2012. The resultant AC and RL trading trajectories are then *executed* on each day at the specified trading time H in the *testing set* from 1-Jul-2012 to 20-Dec-2012. The *implementation shortfall* for both models is calculated and the difference recorded. This allows us to construct a distribution of *implementation shortfall* for each of the AC and RL models, and for all trading hours $H = 9, 10, ..., 16$.

C. Results

Table 1 shows the average % improvement in median *implementation shortfall* for the complete set of stocks and parameter values. These results suggest that the model is more effective for shorter trading horizons ($T = 4$), with an average improvement of up to 10.3% over the base AC model. This result may be biased due to the assumption of order book resilience. Indeed, the efficacy of the trained Q-matrix may be less reliable for stocks which exhibit slow order book resilience, since permanent price effects would affect the state space transitions. In future work, we plan to relax this order book resilience assumption and incoporate permanent effects into state transitions.

Figure 1 illustrates the improvement in median post-trade *implementation shortfall* when executing the volume trajectories generated by each of the models, for each of the candidate stocks at the given trading times. In general, the RL model is able to improve (lower) ex-post *implementation shortfall*, however the improvement seems more significant for early morning/late afternoon trading hours. This could be due to the increased trading activity at these times, resulting in more state-action visits in the *training set* to refine the associated Q-matrix values. We also notice more dispersed performance between 10:00 and 11:00. This time period coincides with the UK market open, where global events may drive local trading activity and skew results, particularly since certain SA stocks are dual-listed on the London Stock Exchange (LSE). The improvement in *implementation shortfall* ranges from 15 bps (85.3%) for trading 1000 000 of SBK between 16:00 and 17:00, to -7 bps (-83.4%) for trading 100 000 SAB between 16:00 and 17:00. Overall, the RL model is able to improve *implementation shortfall* by 4.8%.

Figure 2 shows the % of *correct actions* implied by the Q-matrix, as it evolves through the training process after each tuple visit. Here, a *correct action* is defined as a reduction (addition) in the volume-to-trade based on the max Q-value action, in the case where *spreads* are above (below) the 50%ile and *volumes* are below (above) the 50%ile level. This coincides with the intuitive behaviour we would like the RL agent to learn. These results suggest that finer state granularity ($I, B, W = 10$) improves the overall accuracy of the learning agent, as demonstrated by the higher % *correct actions* achieved. All model configurations seem to converge to some *stationary* accuracy level after approximately 1000 tuple visits,

Parameters			Trading Time(hour)								Average
V	T	I,B,W	9	10	11	12	13	14	15	16	
100000	4	5	23.9	-1.4	4.7	13.4	1.8	3.3	1.8	35.1	10.3
100000	8	5	25.3	4.3	8.3	2.3	1.4	9.9	-0.6	-1.9	6.1
100000	12	5	32.7	-25.2	7.2	-2.7	-1.5	4.6	4.5	-3.3	2.1
1000000	4	5	23.3	-1.3	4.8	9.3	1.9	3.5	1.8	35.0	9.8
1000000	8	5	28.8	5.6	8.2	1.9	1.4	9.9	-0.3	-2.6	6.6
1000000	12	5	33.1	-25.0	7.2	-4.0	-0.8	4.8	4.8	1.2	2.7
100000	4	10	22.9	1.3	3.0	9.7	2.7	5.8	3.5	-26.1	2.8
100000	8	10	26.0	4.3	6.7	-0.2	3.5	8.6	1.6	-3.1	5.9
100000	12	10	27.8	-21.9	7.5	-4.1	0.6	1.8	6.2	-9.5	1.1
1000000	4	10	22.6	1.4	3.1	9.3	2.5	6.0	3.6	-26.1	2.8
1000000	8	10	26.3	5.0	7.2	-0.5	3.3	7.0	2.3	-1.8	6.1
1000000	12	10	27.9	-24.3	8.3	-6.9	0.5	1.8	7.5	-3.3	1.4

TABLE I: Average % improvement in median *implementation shortfall* for various parameter values, using AC and RL models. Training H-dependent.

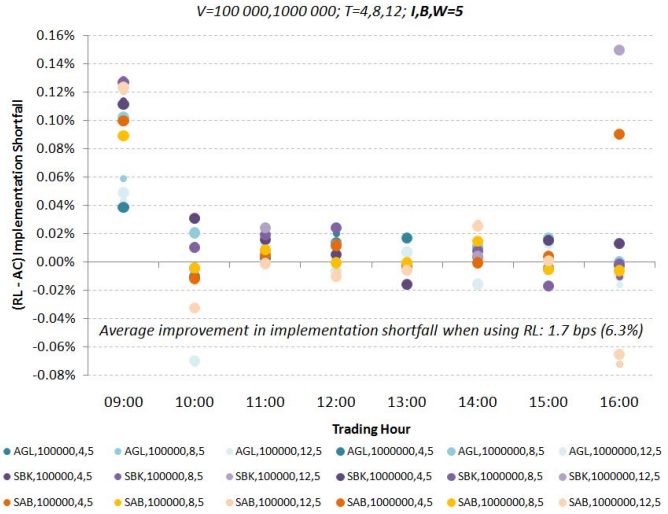


Fig. 1: Difference between median implementation shortfall generated using RL and AC models, with given parameters ($I,B,W = 5$). Training H-dependent.

suggesting that a shorter training period may yield similar results. We do however note that improving the % of correct actions by increasing the granularity of the state space does not necessarily translate into better model performance. This can be seen by Table 1, where the results where $I, B, W = 10$ do not show any significant improvement over those with $I, B, W = 5$. This suggests that the market dynamics may not be fully represented by *volume* and *spread* state attributes, and alternative state attributes should be explored in future work to improve ex-post model efficacy.

Table 2 shows the average standard deviation of the resultant *implementation shortfall* when using each of the AC and RL models. Since we have not explicitly accounted for *variance of execution* in the RL reward function, we see that the resultant trading trajectories generate a higher standard deviation compared to the base AC model. Thus, although the RL model provides a performance improvement over the AC model, this is achieved with a higher degree of execution risk, which may not be acceptable for the trader. We do note that the RL model exhibits comparable risk for $T = 4$, thus validating the use of the RL model to reliably improve IS over short trade horizons. A future refinement on the RL model should incorporate *variance of execution*, such that it is consistent with the AC objective function. In this way, a true comparison of the techniques can be done, and one can conclude as to

Authorized licensed use limited to: Columbia University Libraries. Downloaded on October 23,2026 at 22:57:41 UTC from IEEE Xplore. Restrictions apply.

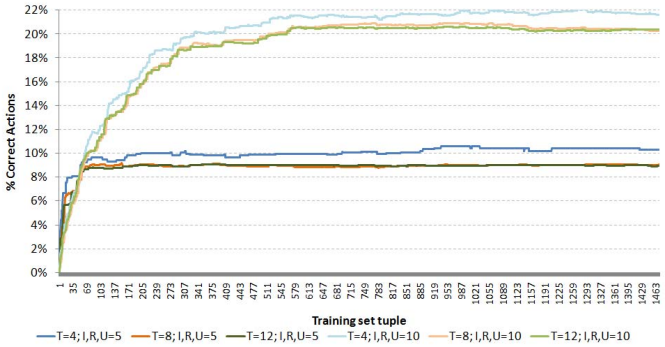


Fig. 2: % correct actions implied by Q-matrix after each training set tuple. Training H-dependent.

Parameters			Standard Deviation(%)		% improvement
V	T	I,B,W	AC	RL	in IS
100000	4	5	0.13	0.17	10.3
100000	8	5	0.14	0.23	6.1
100000	12	5	0.14	0.26	2.1
1000000	4	5	0.13	0.17	9.8
1000000	8	5	0.14	0.23	6.6
1000000	12	5	0.14	0.26	2.7
100000	4	10	0.13	0.17	2.8
100000	8	10	0.14	0.22	5.9
100000	12	10	0.14	0.26	1.1
1000000	4	10	0.13	0.17	2.8
1000000	8	10	0.14	0.22	6.1
1000000	12	10	0.14	0.26	1.4
Average			0.14	0.22	4.8

TABLE II: Standard deviation(%) of implementation shortfall when using AC vs RL models.

whether the RL model indeed outperforms the AC model at a statistically significant level.

V. Conclusion

In this paper, we introduced reinforcement learning as a candidate machine learning technique to *enhance* a given optimal liquidation volume trajectory. Nevmyvaka, Feng and Kearns showed that reinforcement learning delivers promising results where the learning agent is trained to choose the optimal limit order price at which to place the remaining inventory, at discrete periods over a fixed liquidation horizon [20]. Here, we show that reinforcement learning can also be used successfully to modify a given volume trajectory based on market attributes, executed via a sequence of *market orders* based on the prevailing limit order book.

Specifically, we showed that a simple look-up table *Q-learning* technique can be used to train a learning agent to modify a static Almgren-Chriss volume trajectory based on prevailing spread and volume dynamics, assuming order book resiliency. Using a sample of stocks and trade sizes in the South African equity market, we were able to reliably improve post-trade *implementation shortfall* by up to 10.3% on average for short trade horizons, demonstrating promising potential applications of this technique. Further investigations include incorporating *variance of execution* in the RL reward function, relaxing the order book resiliency assumption and alternative state attributes to govern market dynamics.

Acknowledgment

The authors thank Dr Nicholas Westray for his contribution in the initiation of this work, as well as the insightful comments from the anonymous reviewers. This work is based on the research supported in part by the National Research Foundation of South Africa (Grant Number CPRR 70643)

References

[1] R. Almgren, N. Chriss. *Optimal execution of portfolio transactions*, Journal of Risk, 3, pp. 5-40, 2000.
[2] R. Almgren. *Optimal execution with nonlinear impact functions and trading-enhanced risk*, Applied Mathematical Finance, 10(1), pp. 1-18, 2003.
[3] A. Admati, P Pfleiderer. *A theory of intraday patterns: volume and price variability*, Review of Financial Studies, 1(1), pp. 3-40, 1988.
[4] A. Barto, S. Mahadevan. *Recent advances in hierarchical reinforcement learning*, Discrete Event Dynamic Systems, 13(4), pp. 341-379, 2003.
[5] R. Bellman. *The theory of dynamic programming*, Bulletin of the American Mathematical Society, 1954.
[6] R. Bellman, S. Dreyfus. *Applied dynamic programming*, Princeton University Press, Princeton, New Jersey, 1962.
[7] D. Bertsimas, A. Lo. *Optimal control of execution costs*, Journal of Financial Markets, 1(1), pp. 1-50, 1998.
[8] W. Brock, A Kleidon. *Periodic market closure and trading volume: a model of intraday bids and asks*, Journal of Economic Dynamics and Control, 16(3), pp. 451-489, 1992.
[9] L. Chan, J Lakonishok. *The behavior of stock prices around institutional trades*, Journal of Finance, 50(4), pp.1147-1174, 1995.
[10] P. Dayan, C. Watkins. *Reinforcement learning*, Encyclopedia of Cognitive Science, 2001.
[11] H. Degryse, F. deJong, M. Ravenswaaij, G. Wuyts, *Aggressive orders and the resiliency of a limit order market*, Review of Finance, 9(2), pp. 201-242, 2003.
[12] T. Dietterich. *Hierarchical reinforcement learning with the MAXQ value function decomposition*, Abstraction, Reformulation and Approximation, pp. 26-44, 2000.
[13] B. Du Preez. *JSE Market Microstructure*, MSc Dissertation, University of the Witwatersrand, School of Computational and Applied Mathematics, 2013.
[14] F. Garcia, S. Ndiaye. *A learning rate analysis of reinforcement learning algorithms in finite-horizon*, Proceedings of the 15th International Conference on Machine Learning, 1998.
[15] A. Gosavi. *Reinforcement learning: a tutorial survey and recent advances*, INFORMS Journal on Computing, 21(2), pp. 178-192, 2009.
[16] R. Holthausen, R. Leftwich, D. Mayers. *Large-block transactions, the speed of response and temporary and permanent stock-price effects*, Journal of Financial Economics, 26(1), pp. 71-95, 1990.
[17] G. Huberman, W. Stanzl. *Optimal liquidity trading*, Yale School of Management, Working Paper, 2001.
[18] L. Kaelbling, M. Littman, A. Moore. *Reinforcement learning: a survey*, Journal of Artificial Intelligence Research, 4, pp. 237-285, 1996.
[19] J. McCulloch. *Relative volume as a doubly stochastic binomial point process*, Quantitative Finance, 7(1), pp. 55-62, 2007.
[20] Y. Nevmyvaka, Y. Feng., M. Kearns. *Reinforcement learning for optimal trade execution*, Proceedings of the 23rd international conference on machine learning, pp. 673-680, 2006.
[21] A. Perold, *The implementation shortfall: paper vs reality*, Journal of Portfolio Management, 14(3), pp. 4-9, 1988.
[22] M. Puterman, *Markov Decision Processes*, John Wiley and Sons, New York, 1994.
[23] S. Ross. *Introduction to stochastic dynamic programming*, Academic Press, New York, 1983.
[24] S. Singh, T Jaakola, M. Littman, C. Szepesvari. *Convergence results for single-step on-policy reinforcement learning algorithms*, Machine Learning, 38(3), pp. 287-308, 2000.
[25] R. Sutton, A. Barto. *Reinforcement learning*, Cambridge, MA: MIT Press, 1998.
[26] D. Vayanos. *Strategic trading in a dynamic noisy market*, Journal of Finance, 56(1), pp. 131-171, 2001.
[27] C. Watkins. *Learning from delayed rewards*, PhD Thesis, Cambridge University, 1989.

Authorized licensed use limited to: Columbia University Libraries. Downloaded on October 23,2026 at 22:57:41 UTC from IEEE Xplore. Restrictions apply.